BZ205 Konu Anlatımı

İkili Ağaçlar ve İkili Arama Ağaçları

Hocanın ders videoları, slaytları ve sınav soruları temel alınmıştır

27 Ekim 2026

6 İkili Ağaçlar ve İkili Arama Ağaçları

6.1 Neden ağaç?

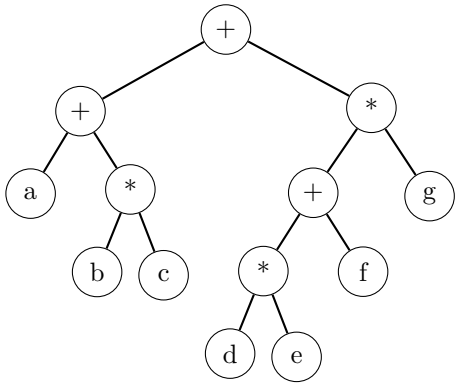
Bağlantılı listede arama, ekleme, silme doğrusaldır ($O(n)$). Hedef: bu işlemleri $O(\log n)$'e indiren bir yapı. Ağaç düğümlerden oluşur: **kök** (ebeveyni yok), **çocuk/ebeveyn**, **yaprak** (çocuğu yok), **kardeş** (aynı ebeveyn), **yol uzunluğu** (kenar sayısı), **derinlik** (kökten düğüme yol), **yükseklik** (düğümden en uzak yaprağa yol; yapraklar 0, ağacın yüksekliği = kökün yüksekliği). **İkili ağaç:** her düğümün en fazla iki çocuğu vardır.

6.2 İfade ağaçları

Yapraklar sayı/değişken, iç düğümler işlemdir. *"Burada önemli husus: sayılar kesinlikle yaprakta olacak"* [video 13 00:03]. Toplama/çarpmada sol–sağ yer değiştirebilir ama *çıkarma ve bölmede sıra önemlidir.*

- **Sonek ifadesinden ağaç:** soldan sağa okunur; işlenenler beklenir, işlem gelince son iki alt ağaç onun sol ve sağ çocuğu olur (yığın mantığı). Örnek `a b c * + d e * f + g * +`: `bc*`→`a+(b*c)`→`de*`→`(d*e)+f`→`((d*e)+f)*g`→ kök `+`.
- **Eşitlikten ağaç:** parantez ve işlem önceliğine dikkat edilerek en içten dışa doğru alt ağaçlar kurulur.

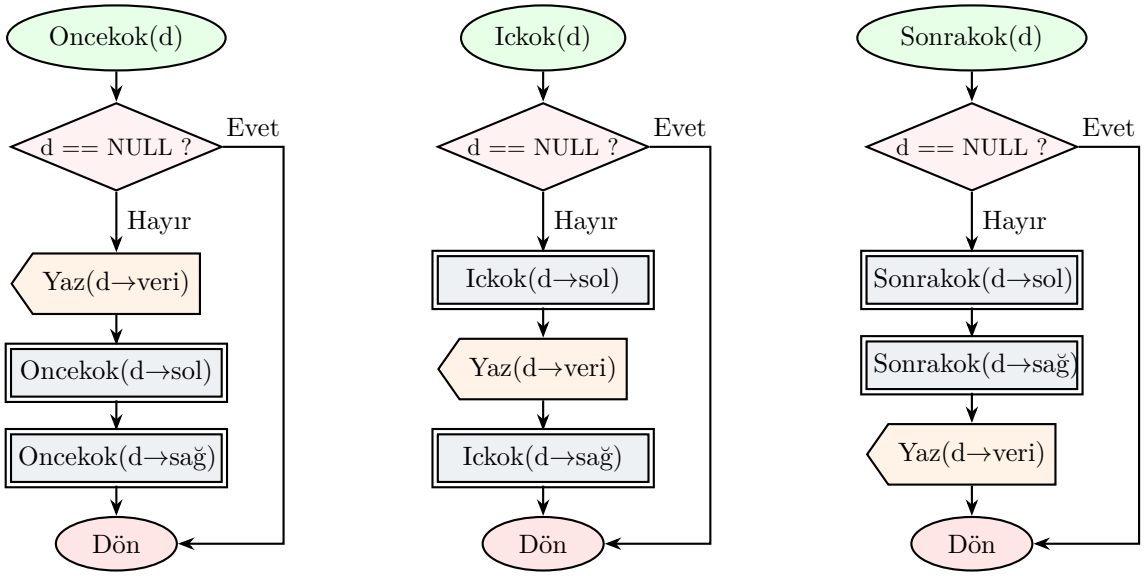
6.3 Ağaç gezme (traversal)



Gezme	Sıra	Sonuç
Öncekök (preorder)	kök, sol, sağ	`+ + a * b c * + * d e f g` (önek)
İçkök (inorder)	sol, kök, sağ	`a + b * c + d * e + f * g` (içek)
Sonrakök (postorder)	sol, sağ, kök	`a b c * + d e * f + g * +` (sonek)

```
Oncekok(d):  if d == NULL return;  Yaz(d);  Oncekok(d->sol);  Oncekok(d->sag)
Ickok(d):    if d == NULL return;  Ickok(d->sol);  Yaz(d);  Ickok(d->sag)
Sonrakok(d): if d == NULL return;  Sonrakok(d->sol);  Sonrakok(d->sag);  Yaz(d)
```

Akış diyagramı: öncekök, içkök, sonrakök



Üç diyagram aynıdır; sadece "Yaz" kutusunun yeri değişir: başta (kök–sol–sağ), ortada (sol–kök–sağ), sonda (sol–sağ–kök).

▷ Bu bölümün adım adım oynatılabilen etkileşimli animasyonu HTML sürümündedir.

"Sol sağ kök. Solun sağa her zaman önceliği var" [video 12 20:19]. Her gezme her düğümü bir kez ziyaret eder: $\Theta(n)$. 2025 vizesinde sorulan "sol taraf → sağ taraf → düğüm" gezmesi sonrakök'tür.

6.4 İkili arama ağacı (İAA)

Kural: her X düğümü için sol alt ağaçtaki tüm anahtarlar X'ten küçük, sağ alt ağaçtakiler büyüktür – *"bu ikili ağacın her aşamasında, her kademede geçerli olacak"*. Harflerde alfabetik önde olan soldadır. Eşit değer için bir kural belirlenip (ör. sağa) ona sadık kalınır.

- **Arama:** kökten başla; aranan küçükse sola, büyükse sağa; NULL'a gelinirse yok. Dengeli ağaçta $O(\log n)$, ip gibi dizilmiş ağaçta $O(n)$.
- **En küçük / en büyük:** sürekli en sola / en sağa gidilir (*"NULL'ı gördüğüm yerdeki bir önceki eleman en küçük"*).
- **İçkök gezme sıralı listeyi verir** – ayrıca sıralama algoritması gerekmez.
- **Ekleme:** arama gibi inilir, NULL bulunan yere yaprak olarak eklenir. Sıralı gelen veri (1,2,3,…) ağacı sağa doğru "ip gibi" uzatır.
- **Silme (hocanın kuralları):**
  1. Yaprak ⇒ doğrudan silinir.
  2. Tek çocuklu ⇒ çocuğu, silinen düğümün ebeveynine (dedeye) bağlanır.
  3. İki çocuklu ⇒ **sağ alt ağacın en küçüğü (minimumu) silinen düğümün yerine geçer** [video 12 43:31]; sonra o eleman eski yerinden (1. veya 2. kurala göre) silinir.

! Hoca derste vurguladı WhatsApp aktarımı

"AVL'de kök sildikten sonra yeni kökü nasıl seçiyorduk? – Sağ alt ağacın minimumu." "İkisi de mantıken doğru ama biz sağın en küçüğünü alarak işleyeceğiz dedi hoca." "Listeyi ikili arama

ağacına yerleştirirken... yine de tüm adımları tek tek gösterin dedi."

Örnek: KAR, MOR, BAL, ŞEN, ÜZÜM, TEK, CAN, ÖZ ekle; kökü sil; DAL ekle

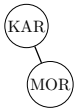
▷ Bu bölümün adım adım oynatılabilen etkileşimli animasyonu HTML sürümündedir.

Ekleme adımları (küçükler sola, büyükler sağa; her kademede kontrol):

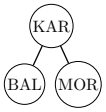
KAR eklendi



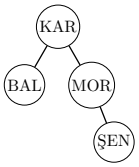
MOR eklendi



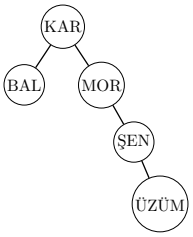
BAL eklendi



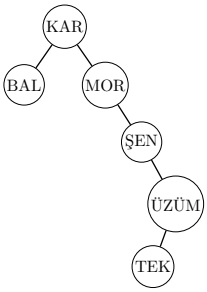
ŞEN eklendi



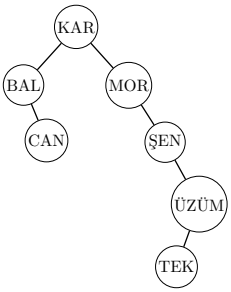
ÜZÜM eklendi



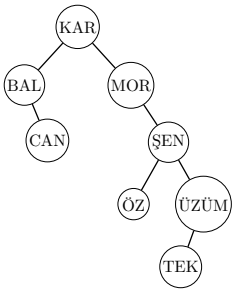
TEK eklendi



CAN eklendi

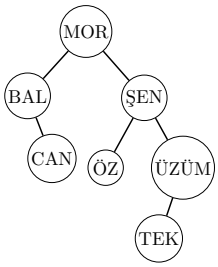


ÖZ eklendi

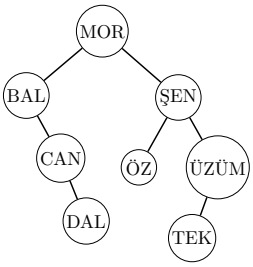


1. kök silme (**KAR**)

KAR iki çocuklu: sağ alt ağacın en küçüğü MOR yerine geçer.



DAL eklendi



6.5 İAA kodları

```
Node* Ara(Node* d, x)
   if (d == NULL || d->veri == x) return d
   if (x < d->veri) return Ara(d->sol, x)
   return Ara(d->sag, x)

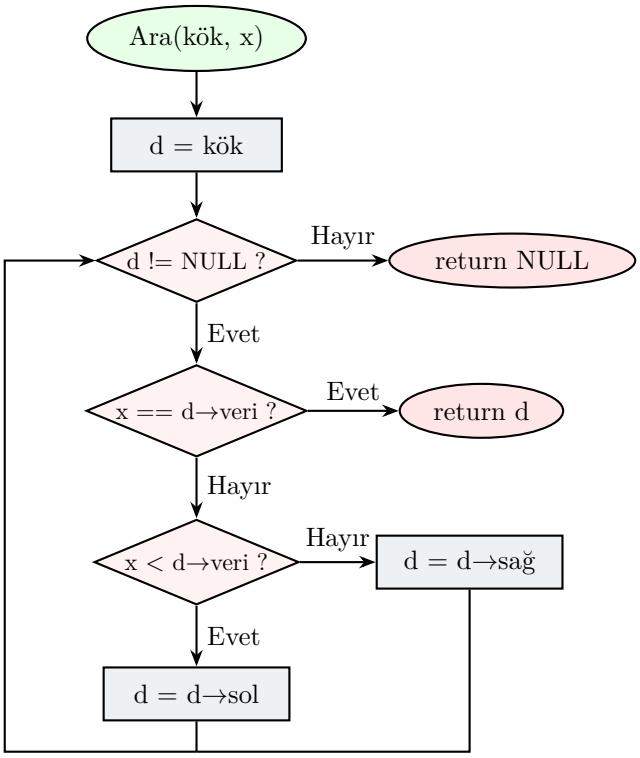
Node* Ekle(Node* d, x)
   if (d == NULL) return YeniDugum(x)
   if (x < d->veri)  d->sol = Ekle(d->sol, x)
```

```
    else              d->sag = Ekle(d->sag, x)
    return d

Node* EnKucuk(Node* d)   while (d->sol != NULL) d = d->sol;   return d
```

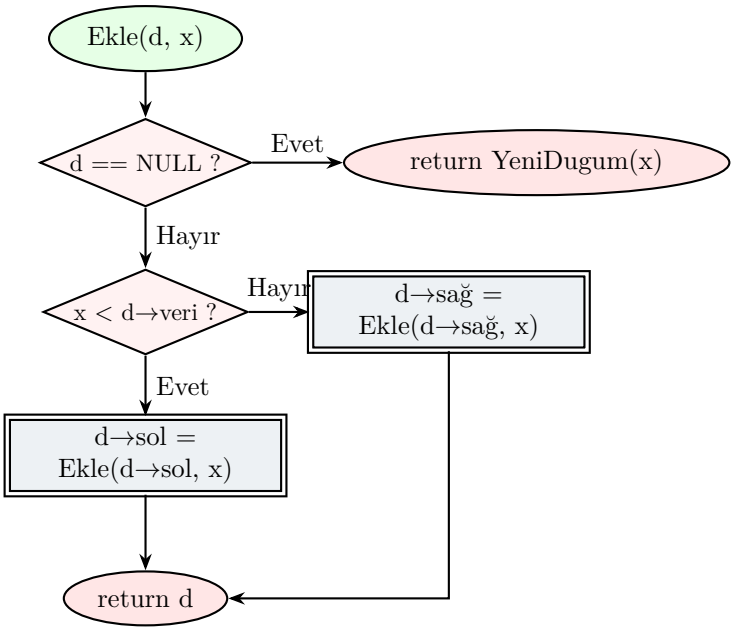
Akış diyagramı: ikili arama ağacında arama (hocanın ödevi)

Hocanın derste verdiği soru: *"ikili ağaçta verilen bir sayıyı arayan fonksiyonun akış diyagramı"*.



Akış diyagramı: ikili arama ağacına ekleme

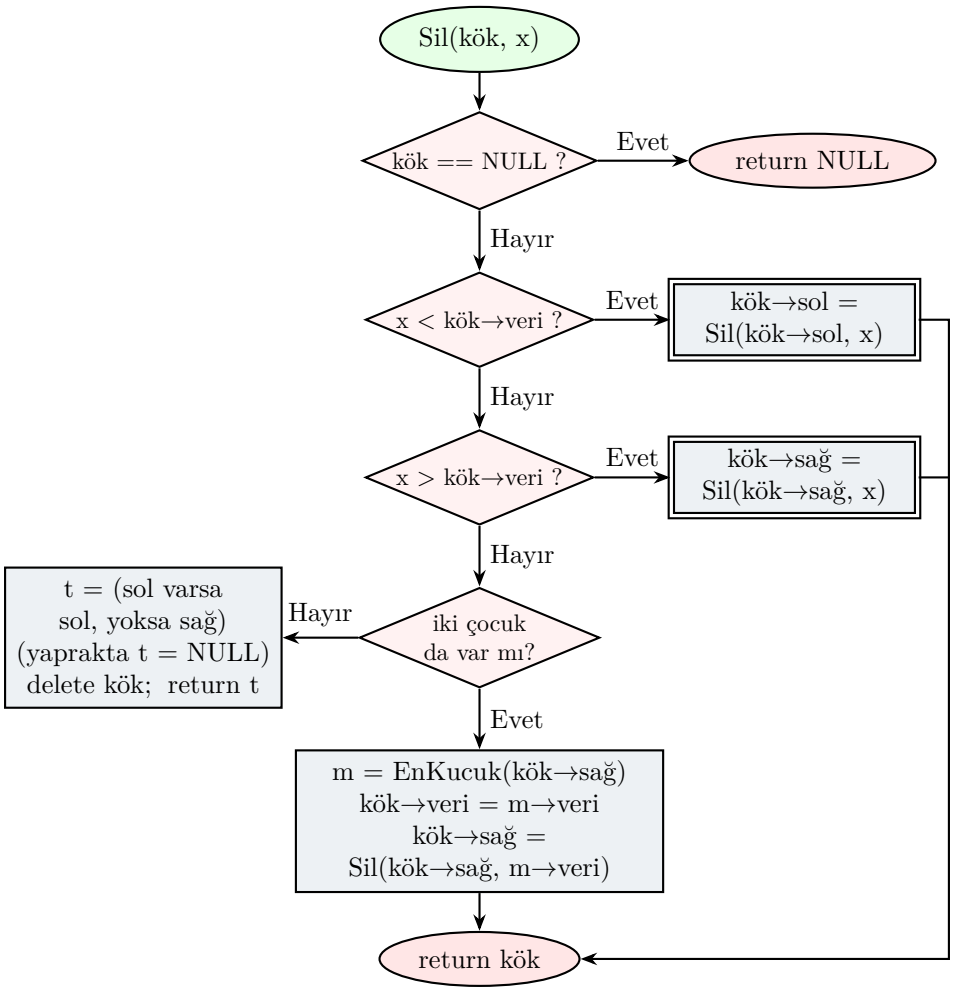
Eşit değerler için seçilen kural (burada: sağa) sonuna kadar uygulanır.



Akış diyagramı: ikili arama ağacından silme (2025–26 vize sorusu)

```
Sil(kok, x)
  if (kok == NULL) return NULL
  if (x < kok->veri)
     kok->sol = Sil(kok->sol, x)
  else if (x > kok->veri)
     kok->sag = Sil(kok->sag, x)
  else if (iki cocugu var)
     m = EnKucuk(kok->sag)
     kok->veri = m->veri
     kok->sag = Sil(kok->sag, m->veri)
  else            // yaprak / tek cocuk
     t = (sol varsa sol, yoksa sag)
     delete kok;  return t
  return kok
```

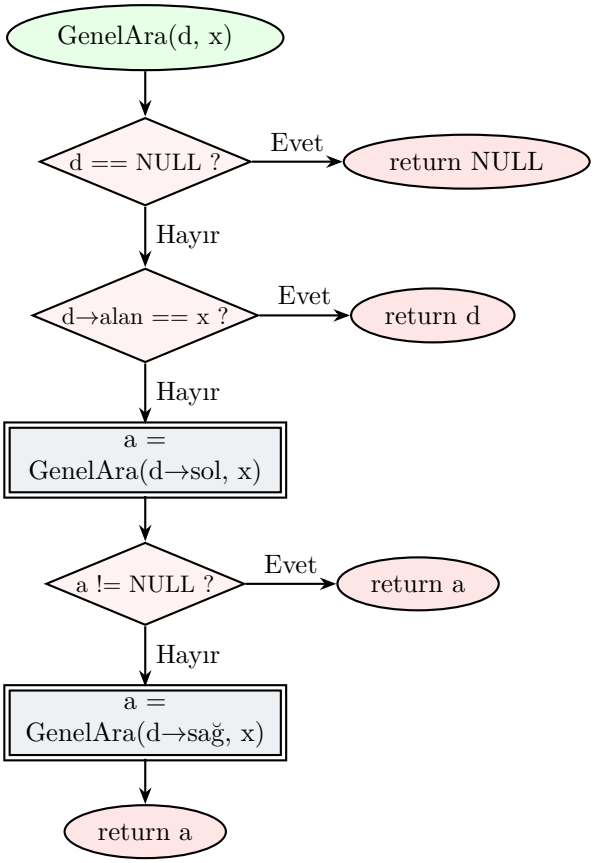
Hocanın kuralı: iki çocukluda *sağ alt ağacın en küçüğü* yerine geçer. Satır satır yürütme zamanı çözümler dosyasındadır (2025–26 vize).



Genel (arama olmayan) ikili ağaçta arama – hocanın tahtada yazdığı

Ağaç aranan alana göre indekslenmemişse (ör. AVL numaraya göre, biz ada göre arıyoruz) hem sol hem sağ alt ağaca bakılır: `if (d==NULL) return NULL; if (d->ad==x) return d; a = Ara(d->sol,x); if (a!=NULL) return a; return Ara(d->sag,x);` – $O(n)$.

Akış diyagramı: indekslenmemiş alana göre ağaçta arama



6.6 Zaman ve alan karmaşıklığı

h: ağacın yüksekliği. Dengeli ağaçta $h \approx \log_2 n$, ip gibi dizilmiş (sıralı veriyle oluşmuş) ağaçta $h = n - 1$.

İşlem	En iyi	En kötü	Ek alan	Not
Arama	$\Omega(1)$	$O(n)$	$O(1)$ / $O(h)$	en iyi kökte; $O(h)$, dengeli ise $O(\log n)$; özyinelemeli sürüm h kadar çağrı
Ekleme	$\Omega(1)$	$O(n)$	$O(h)$	NULL bulunana kadar inilir: $O(h)$
Silme	$\Omega(1)$	$O(n)$	$O(h)$	arama + (iki çocukluda) sağ alt ağacın en küçüğünü bulma: $O(h)$
En küçük / en büyük	$\Omega(1)$	$O(n)$	$O(1)$	sürekli sola / sağa: $O(h)$
Gezmeler (öncekök, içkök, sonrakök)	$\Theta(n)$	$\Theta(n)$	$O(h)$	her düğüm bir kez ziyaret edilir
İndekslenmemiş alanda arama	$\Omega(1)$	$O(n)$	$O(h)$	sol/sağ seçimi yapılamaz, iki alt ağaca da bakılır